

NLP Python Laboratory Guide

Hands-On Implementation of Word2Vec, BiLSTM-CRF & Transformers

Natural Language Processing Laboratory & Python

Implementation Master Guide

Complete PyTorch & HuggingFace Implementations of Word2Vec SGNS, BiLSTM-CRF NER & Fine-Tuned Transformer Classification

Lab Experiment 1: PyTorch Implementation of Word2Vec Skip-Gram with Negative Sampling

```
import torch
import torch.nn as nn
import torch.optim as optim

class SkipGramNegativeSampling(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super(SkipGramNegativeSampling, self).__init__()
        self.target_embeddings = nn.Embedding(vocab_size, embed_dim)
        self.context_embeddings = nn.Embedding(vocab_size, embed_dim)

        # Initialize weights
        nn.init.uniform_(self.target_embeddings.weight, -0.5/embed_dim, 0.5/embed_dim)
        nn.init.constant_(self.context_embeddings.weight, 0)

    def forward(self, target_words, context_words, negative_words):
        # target: [B], context: [B], negative: [B, K]
        v_target = self.target_embeddings(target_words) # [B, d]
        v_context = self.context_embeddings(context_words) # [B, d]
        v_neg = self.context_embeddings(negative_words) # [B, K, d]

        # Positive score: log(sigma(v_c * v_w))
        pos_score = torch.sum(v_target * v_context, dim=1) # [B]
        pos_loss = -torch.log(torch.sigmoid(pos_score) + 1e-10)

        # Negative score: sum_k log(sigma(-v_nk * v_w))
        neg_score = torch.bmm(v_neg, v_target.unsqueeze(2)).squeeze(2) # [B, K]
        neg_loss = -torch.sum(torch.log(torch.sigmoid(-neg_score) + 1e-10), dim=1)

        return torch.mean(pos_loss + neg_loss)

# Model instantiation
model = SkipGramNegativeSampling(vocab_size=10000, embed_dim=128)
print(model)
```

Lab Experiment 2: Transformer Fine-Tuning with HuggingFace PyTorch

```
from transformers import BertTokenizer, BertForSequenceClassification
from torch.utils.data import DataLoader
import torch

# Load pretrained BERT tokenizer and sequence classifier
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)

sentences = ["Natural Language Processing with Transformers is amazing!", "This baseline model produces bad results"]
labels = torch.tensor([1, 0])

inputs = tokenizer(sentences, padding=True, truncation=True, return_tensors="pt")
outputs = model(**inputs, labels=labels)
loss = outputs.loss
logits = outputs.logits

print(f"Training Loss: {loss.item():.4f}")
print("Predicted Class Logits:\n", logits)
```

Lab Experiment 3: BiLSTM-CRF Named Entity Recognition (NER) in PyTorch

```
import torch
import torch.nn as nn

class BiLSTM_CRF(nn.Module):
    def __init__(self, vocab_size, tag_to_ix, embedding_dim, hidden_dim):
        super(BiLSTM_CRF, self).__init__()
        self.embedding_dim = embedding_dim
        self.hidden_dim = hidden_dim
        self.vocab_size = vocab_size
        self.tag_to_ix = tag_to_ix
        self.tagset_size = len(tag_to_ix)

        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim // 2,
                             num_layers=1, bidirectional=True, batch_first=True)
        self.hidden2tag = nn.Linear(hidden_dim, self.tagset_size)

        # Transition matrix: transitions[i, j] = score of transitioning from j to i
        self.transitions = nn.Parameter(torch.randn(self.tagset_size, self.tagset_size))

    def forward(self, sentence):
        embeds = self.embedding(sentence)
        lstm_out, _ = self.lstm(embeds)
        lstm_feats = self.hidden2tag(lstm_out)
        return lstm_feats

tag_to_ix = {"B-PER": 0, "I-PER": 1, "B-LOC": 2, "I-LOC": 3, "O": 4}
model = BiLSTM_CRF(vocab_size=5000, tag_to_ix=tag_to_ix, embedding_dim=100, hidden_dim=128)
print("BiLSTM-CRF Architecture Initialized Successfully!")
```